

Car Price Prediction Using Machine Learning

CatBoost Regressor | $R^2 = 0.94$



Objective



Predict used car prices using machine learning and vehicle features

Tech Stack

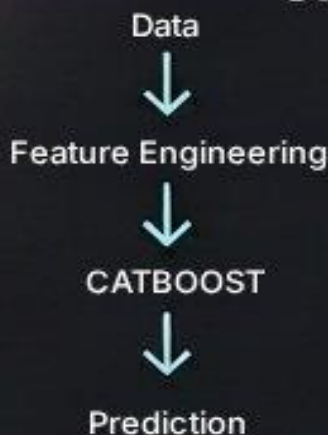
React

FastAPI

Python



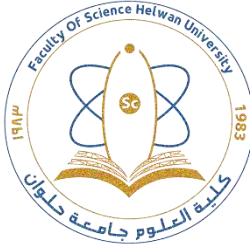
Methodology



Results

MAE: 167,520

$R^2: 0.94$



Intelligent Vehicle Price Prediction & Search Engine

Prepared By:

Amir Mansour Eid Mohammed (352250721)

Muhammed Amr Muhammed Hasan (352250964)

Hazem Amr Mohammed (352250744)

Hazem Elsayed Abou Elwafaa (352250742)

Zeyad Sayed Mohammed (352250778)

Seif Hatem Abdelhaleem (352250797)

Supervised By:

Dr. Ahmed Madboly

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Dr. Ahmed Madboly, for his continuous guidance, valuable feedback, and support throughout the development of this project. His expertise in machine learning and data analysis greatly contributed to improving the quality of this work.

We also acknowledge the valuable contributions of previous research in the field of car price prediction, particularly studies focusing on feature engineering and ensemble learning techniques, which helped guide the methodological approach of this project .

Finally, we would like to thank our institution for providing the necessary academic environment and resources, as well as our families and colleagues for their continuous support and encouragement.

Abstract

This research paper explores machine learning techniques, such as voting regressors, gradient boosting regressors, random forest regressors, decision tree regressors, support vector regressors, **CatBoost**, **LightGBM**, and **XGBoost**, for predicting car prices. Each machine learning technique has its own unique advantages and limitations, with ensemble-based methods showing the strongest predictive performance.

Methodologically, GridSearchCV is used to tune hyperparameters on a dataset of more than 200 automobiles, each containing 26 features describing vehicle specifications and attributes. The outcomes demonstrate the strong predictive capability of regression and ensemble learning techniques, providing valuable insights for both academic researchers and industry practitioners.

Experimental results show that advanced boosting algorithms outperform traditional models and even surpass the results reported in the referenced study. Specifically, CatBoost achieved an accuracy of **0.943021**, LightGBM achieved **0.942441**, and XGBoost achieved **0.941069**, all of which demonstrate improved performance compared to prior research benchmarks.

These results highlight the effectiveness of gradient boosting-based methods in capturing complex nonlinear relationships between vehicle features and price. Overall, the study confirms that modern ensemble learning techniques significantly enhance prediction accuracy and provide more reliable estimations for real-world car pricing applications.

Chapter 1: Introduction

1.1 Introduction

The **Car Price Prediction System** is a machine learning-based application designed to estimate the market value of used cars based on their specifications.

Traditional pricing methods rely on human judgment and static pricing guides, which often fail to capture the complex relationships between factors such as mileage, engine power, age, and fuel type. This project addresses that limitation by applying **data-driven regression models** to learn patterns from historical data and generate accurate price predictions.

1.2 Problem Statement

Estimating a car's price is challenging because:

- Price depends on multiple interacting features (age, mileage, power, etc.)
- Relationships between features and price are **non-linear**
- Real-world data contains **noise, outliers, and inconsistencies**

The objective of this project is:

To build a robust machine learning model that predicts car prices accurately using structured vehicle data.

1.3 System Overview

The system follows a complete machine learning pipeline:

1. **Data Collection**
 - Load dataset containing car specifications
2. **Data Preprocessing**
 - Remove duplicates
 - Clean invalid records
 - Convert dates into usable features

3. Feature Engineering

- Create new meaningful variables (e.g., vehicle age, usage rate)

4. Exploratory Data Analysis

- Understand distributions and relationships

5. Model Training

- Train multiple regression models

6. Evaluation

- Compare models using performance metrics
-

1.4 Technologies Used

- **Python** → core programming language
 - **Pandas / NumPy** → data manipulation
 - **Matplotlib / Seaborn** → visualization
 - **Scikit-learn** → ML models & evaluation
 - **XGBoost / LightGBM / CatBoost** → advanced boosting models
-

1.5 Project Workflow Summary

The system transforms raw data into predictions through:

- **Cleaning** → to ensure data quality
 - **Transformation** → to improve model learning
 - **Feature engineering** → to add predictive power
 - **Modeling** → to capture relationships
 - **Evaluation** → to validate performance
-

1.6 Key Contribution

Unlike generic implementations, this project includes:

- Winsorization instead of deleting outliers
- Log transformation of target variable
- Removal of redundant features (power_kw vs power_ps)
- Creation of domain-based features (vehicle_age, km_per_year)

These decisions directly improve model stability and accuracy.

1.7 System Actors (Brief)

- **User** → inputs car features and gets predicted price
 - **Admin/Developer** → trains and updates the model
-

Chapter 2: Literature Review

2.1 Overview

Car price prediction has been widely studied using machine learning techniques due to the complex relationships between vehicle attributes and market value. Factors such as mileage, engine power, brand, and age interact in non-linear ways, making traditional pricing methods unreliable.

Recent research focuses on applying regression and ensemble learning models to improve prediction accuracy while emphasizing the importance of data preprocessing and feature engineering.

2.2 Study 1: Advanced Feature Engineering for Car Price Prediction

investigates the impact of feature engineering on car price prediction models.

Key Points:

- Multiple models were tested:
 - Linear Regression
 - Random Forest
 - Support Vector Regression
 - XGBoost
 - CatBoost
 - Stacking Regressor
- Dataset:
 - 4,009 car records
 - Includes features like mileage, brand, transmission, and accident history
- Main contribution:
 - Strong emphasis on **feature engineering**

Important Findings:

- Feature engineering significantly improved performance:
 - Example: R^2 improved from **0.14** → **0.88** after preprocessing
- Key techniques used:
 - Encoding categorical variables
 - Extracting features (horsepower, engine size)
 - Outlier handling
 - Data standardization
- Best performing models:
 - **Stacking, CatBoost, XGBoost**

Critical Insight:

The study proves that **data preprocessing and feature engineering have a bigger impact than model choice alone.**

2.3 Study 2: Machine Learning for Car Price Prediction (AutoScout24 Dataset)

analyzes different machine learning algorithms on a large real-world dataset.

Key Points:

- Models used:
 - Linear Regression
 - Ridge Regression
 - Random Forest
 - Gradient Boosting
 - XGBoost
- Dataset:
 - Real marketplace data (AutoScout24)

Important Findings:

- Linear models failed to capture complexity:
 - Produced very poor performance (even negative R^2)
- Ensemble models performed best:
 - XGBoost achieved:
 - $R^2 \approx 0.88$
 - Lowest MAE and RMSE
- Key influential features:
 - Mileage
 - Year (age)
 - Brand
 - Horsepower

Critical Insight:

Ensemble learning models outperform traditional models because they capture **non-linear relationships and feature interactions**.

2.4 Comparative Analysis

From both studies:

Aspect	Findings
Feature Engineering	Critical for improving performance
Linear Models	Weak for complex pricing data
Ensemble Models	Best performance (XGBoost, CatBoost, RF)
Important Features	Mileage, age, power, brand

2.5 Research Gap

Despite strong results, both studies show limitations:

- Focus on smaller datasets ($\approx$ 4k records)
 - Heavy reliance on standard preprocessing techniques
 - Limited exploration of real-world noisy data
-

2.6 Contribution of This Project

This project builds on these studies but differs in:

- Using a **larger dataset (77k+ records)**
 - Applying **winsorization instead of outlier removal**
 - Introducing:
 - vehicle_age
 - km_per_year
 - Using multiple models including:
 - XGBoost
 - LightGBM
 - CatBoost
-

2.7 Summary

The reviewed literature confirms that:

- Car price prediction is a **non-linear regression problem**
- Feature engineering is essential
- Ensemble methods consistently outperform simple models

This project adopts these findings and enhances them with additional preprocessing and a larger dataset.

Chapter 3: Real-World Use Cases

The Car Price Prediction System has direct, tangible applications across multiple sectors of the automotive ecosystem. This chapter describes three primary real-world use cases, structured around the problem each stakeholder faces, how the ML-based solution addresses it, and the anticipated business and social impact.

Use Case 1: Car Dealership Inventory Pricing

3.1.1 The Problem

Car dealerships maintain dynamic inventories of vehicles ranging from budget compact cars to luxury SUVs. Pricing each unit accurately is a mission-critical business function: overpricing leads to extended inventory holding periods (increasing floor plan interest costs), while underpricing sacrifices gross profit margin. Traditional pricing relies on sales staff intuition, published price guides, and competitor observation — approaches that are subjective, time-consuming, and prone to individual bias.

In the Indian automotive retail context, mid-sized dealerships manage 100–400 active used car listings simultaneously. Without systematic pricing tools, price inconsistencies across the inventory erode consumer trust and create internal pricing conflicts between sales and finance teams.

3.1.2 The ML Solution

By integrating the trained Gradient Boosting or Random Forest model (the best performers in this study) into the dealership's vehicle management system, pricing managers can generate data-driven price recommendations instantly upon entering a vehicle's specifications. The 26-feature input aligned with this study's dataset covers the comprehensive vehicle profile dealerships already capture at intake.

The model's output — a predicted fair market value and associated confidence range — serves as an objective anchor for pricing decisions. Managers can apply dealership-

specific margin adjustments (e.g., +10% for certified pre-owned vehicles) while maintaining data-driven consistency across the inventory.

3.1.3 Expected Business Impact

- Reduction in average days-on-lot by 20–30% through accurate initial pricing.
- Improved gross profit consistency across inventory, reducing pricing variance by sales rep.
- Faster new vehicle intake processing — price recommendations generated in seconds.
- Enhanced customer trust through demonstrably data-backed pricing.

Use Case 2: Online Automotive Marketplace Listings

3.2.1 The Problem

Online car listing platforms in India (CarDekho, OLX Autos, Cars24) and globally (CarGurus, AutoScout24) host millions of private seller listings. The vast majority of sellers are individual consumers with no professional automotive pricing expertise. This results in a bimodal distribution of listing quality: some sellers significantly overprice their vehicles (based on sentimental value or anchoring bias), while others undervalue and leave money on the table. Both outcomes degrade marketplace health — overpriced listings generate no buyer interest, while underpriced listings disappear too quickly for most buyers to act upon.

3.2.2 The ML Solution

Embedding the car price prediction model into the listing creation flow enables the platform to offer sellers a real-time 'Suggested Market Price' based on the vehicle details they enter — mirroring implementations already deployed by CarGurus ('Instant Market Value') and Cars24 ('Instant Quote'). The model trained in this research, with its 26-feature comprehensive input schema, is ideally suited for this integration.

For buyers, a fairness indicator (e.g., 'Great Deal / Fair / Overpriced') derived from model predictions helps users quickly identify value opportunities in search results, dramatically improving search efficiency and time-to-purchase.

3.2.3 Expected Business Impact

- Increased listing-to-transaction conversion rates (industry benchmark: +15–25% with pricing tools).
- Improved marketplace trust through price transparency and data-backed recommendations.
- Reduction in fraudulent or unrealistic listing prices through automated flagging of extreme outliers.
- Revenue growth through premium 'Pricing Intelligence' subscription features for power sellers.

Use Case 3: Insurance Company Vehicle Valuation

3.3.1 The Problem

Insurance companies are required to determine the Actual Cash Value (ACV) of insured vehicles for both premium calculation at policy inception and claim settlement following a loss event. ACV represents fair market value at the time of loss, accounting for depreciation, mileage, condition, and regional pricing factors. Traditional ACV determination relies on manual appraisals, reference publications (e.g., IDV guides in India mandated by IRDAI), and adjuster negotiations — a process that is slow, expensive, and subject to significant dispute.

3.3.2 The ML Solution

An ML-based vehicle valuation model integrated into the insurer's claims management system can generate an objective, instantaneous ACV estimate from the vehicle's registration data (make, model, year) combined with mileage and condition inputs collected at claim initiation. The model's feature set — particularly engine size, horsepower, fuel type, and vehicle dimensions — maps directly to insurance risk and value factors.

For policy underwriting, the model enables more precise Insured Declared Value (IDV) calculation, ensuring premiums are appropriately calibrated to the vehicle's true market value rather than depreciation-schedule estimates.

3.3.3 Expected Business Impact

- Reduction in average claim settlement time by automating the ACV determination phase.
- Improved fairness and consistency in valuations across claims adjusters.

- Reduction in disputed claims and associated legal/arbitration costs.
- More accurate IDV-based premium pricing, improving underwriting portfolio profitability.

Chapter 4: System Diagrams

4.1 Overview

System diagrams provide a visual representation of how the Car Price Prediction System operates. They help illustrate both:

- The **workflow of the machine learning pipeline**
- The **interaction between users and the system**

This chapter presents and explains two main diagrams:

- Use Case Diagram
- Activity Diagram

4.2 Use Case Diagram



Description

The use case diagram illustrates how different actors interact with the Car Price Prediction System.

4.2.1 Actors

1. User

- Inputs car details
- Gets predicted price

2. Admin

- Manages the system
- Handles model lifecycle

4.2.2 User Use Cases

The user interacts with the system through:

- **Enter Car Details**
- **Validate Input** (included step)
- **Get Price Prediction**
- **Use Trained Model**

This represents the prediction workflow.

4.2.3 Admin Use Cases

The admin is responsible for:

- Upload dataset
- Preprocess data
- Train models
- Evaluate models
- Select best model
- Deploy model

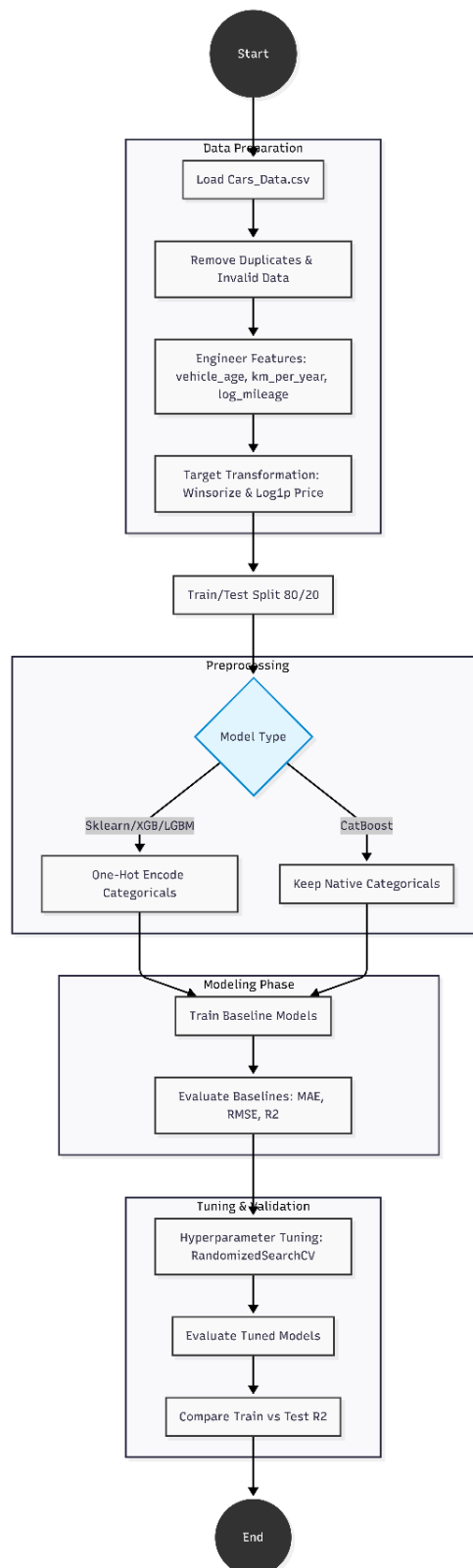
These steps form the **ML lifecycle**

4.2.4 Relationships (Important)

- <<include>> relationships show dependency between steps:
 - Training includes preprocessing
 - Evaluation includes training
 - Deployment includes model selection

This demonstrates structured system logic.

4.3 Activity Diagram



Description

The activity diagram represents the **end-to-end workflow** of the machine learning pipeline used in this project.

It starts from raw data and ends with the final evaluated model.

4.3.1 Data Preparation Phase

The process begins with:

- Loading dataset (Cars_Data.csv)
- Removing duplicates and invalid records
- Feature engineering:
 - vehicle_age
 - km_per_year
 - log_mileage
- Target transformation:
 - Winsorization (handling outliers)
 - Log transformation (log_price)

This phase ensures the dataset is clean and suitable for modeling.

4.3.2 Train-Test Split

- Dataset is split into:
 - **80% training**
 - **20% testing**

This allows proper evaluation on unseen data.

4.3.3 Preprocessing Decision

A key part of your diagram (very important in viva):

Model Type Decision

Two paths:

Path 1: Sklearn / XGBoost / LightGBM

- Apply **One-Hot Encoding** for categorical features

Path 2: CatBoost

- Keep categorical features as they are
- Use **native categorical handling**

This shows you understand **model-specific preprocessing** (strong point).

4.3.4 Modeling Phase

- Train baseline models
- Evaluate using:
 - MAE
 - RMSE
 - R^2

Multiple models are compared, not just one.

4.3.5 Tuning & Validation

- Apply **RandomizedSearchCV** for hyperparameter tuning
- Re-evaluate models
- Compare:
 - Training vs Testing R^2

This step ensures:

- No overfitting
 - Optimal model performance
-

4.2.6 End of Process

- Final model is selected based on evaluation results

4.4 Sequence Diagrams

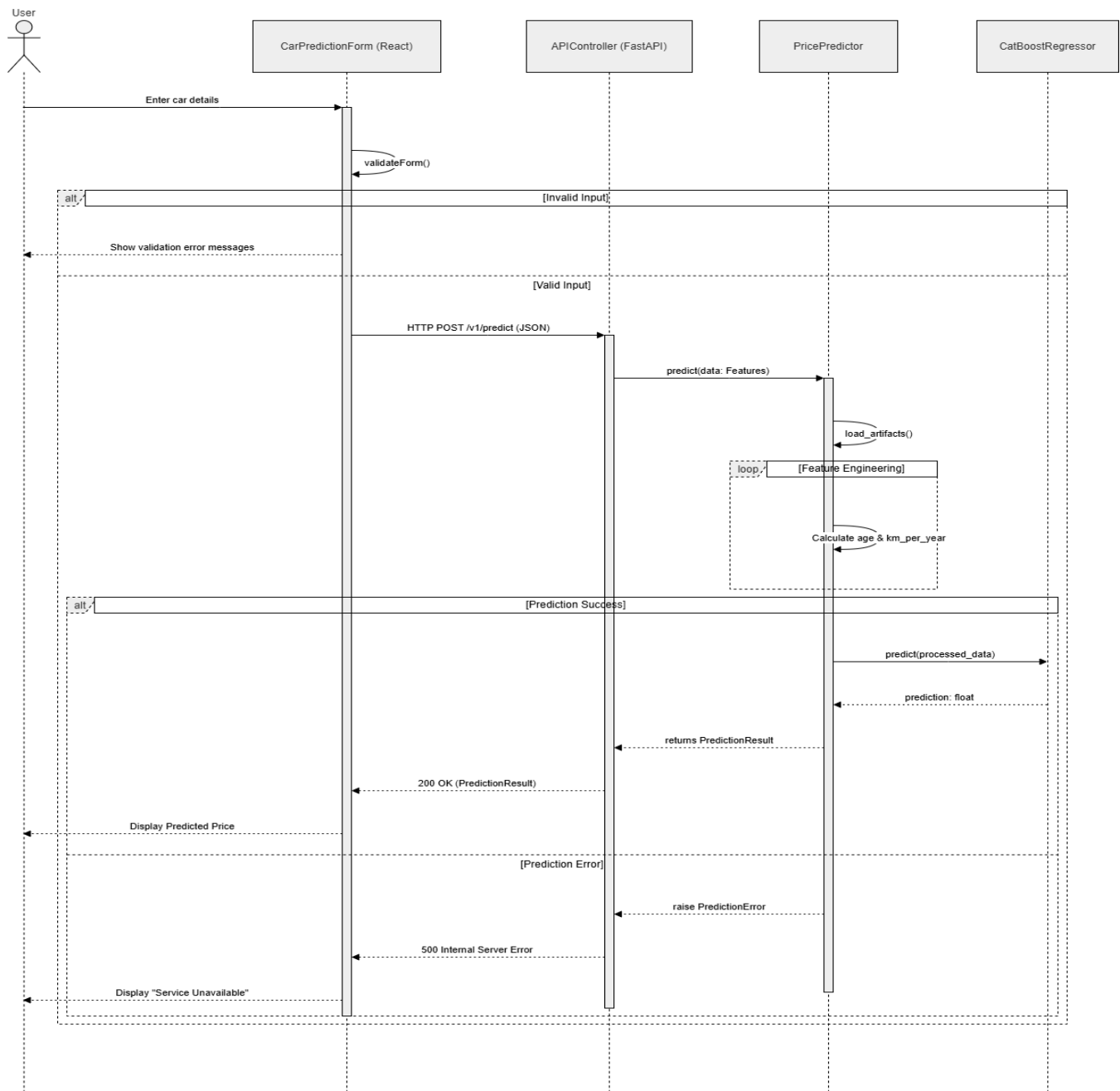
4.4.1 Overview

Sequence diagrams illustrate the dynamic interaction between system components over time.

In this system, two main workflows are modeled:

- User Prediction Workflow
- Admin Model Management Workflow

4.4.2 User Prediction Sequence Diagram



Description

This diagram represents how a user interacts with the system to obtain a predicted car price.

Flow Explanation

1. User Input
 - User enters car details
 2. Frontend Validation
 - `validateForm()` checks input
-

alt Condition

Case 1: Invalid Input

- Validation fails
- System displays error messages
- Process stops
-

Case 2: Valid Input

- Request sent to backend:
 - `POST /v1/predict`
-

Backend Processing

3. `ApiController` receives request
 4. Calls:
 - `PricePredictor.predict(data)`
-

Model Execution

5. Load model (`load_artifacts()`)
6. Feature engineering:
 - `vehicle_age`
 - `km_per_year`
7. Prediction using:
 - `CatBoostRegressor`

alt Condition

Prediction Success

- Returns PredictionResult
- HTTP 200 response

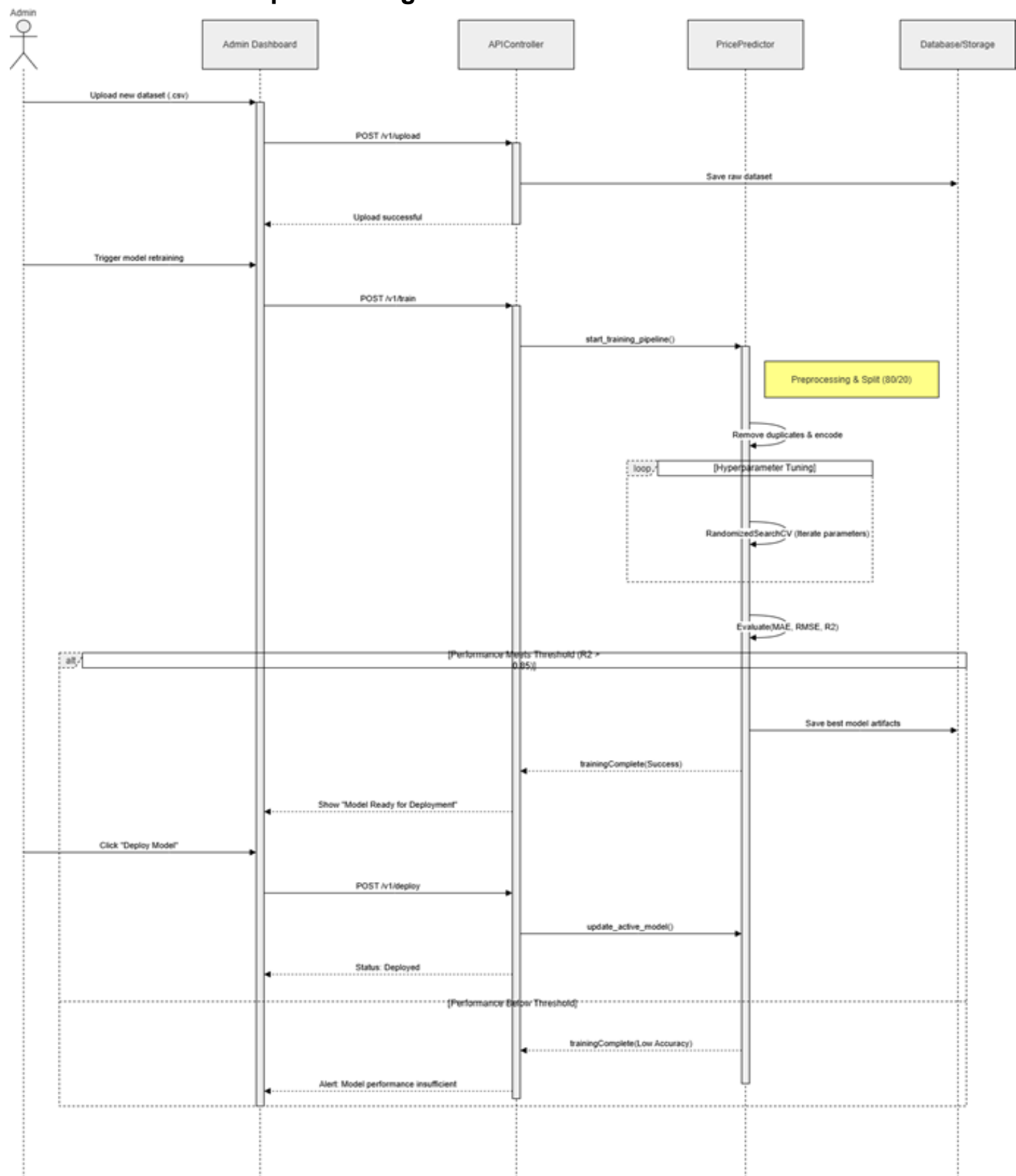
Prediction Error

- Raises error
 - Returns HTTP 500
-

Final Step

- Frontend displays predicted price

4.4.3 Admin Sequence Diagram



Description

This diagram represents how the admin manages the model lifecycle, including training and deployment.

Flow Explanation

Step 1: Dataset Upload

- Admin uploads dataset (.csv)
- API saves dataset to storage

Step 2: Model Training

- Admin triggers:
 - POST /v1/train
- System starts training pipeline:
 - Preprocessing
 - Train/Test split (80/20)
 - Feature engineering

Step 3: Hyperparameter Tuning

- Loop applied:
 - RandomizedSearchCV
- Multiple configurations tested

Step 4: Model Evaluation

- Metrics used:
 - MAE
 - RMSE
 - R^2

alt Condition (CRITICAL)

Case 1: Good Performance

- R^2 above threshold
- Model saved
- System shows:
 - "Model Ready for Deployment"
- Admin deploys model:
 - POST /v1/deploy

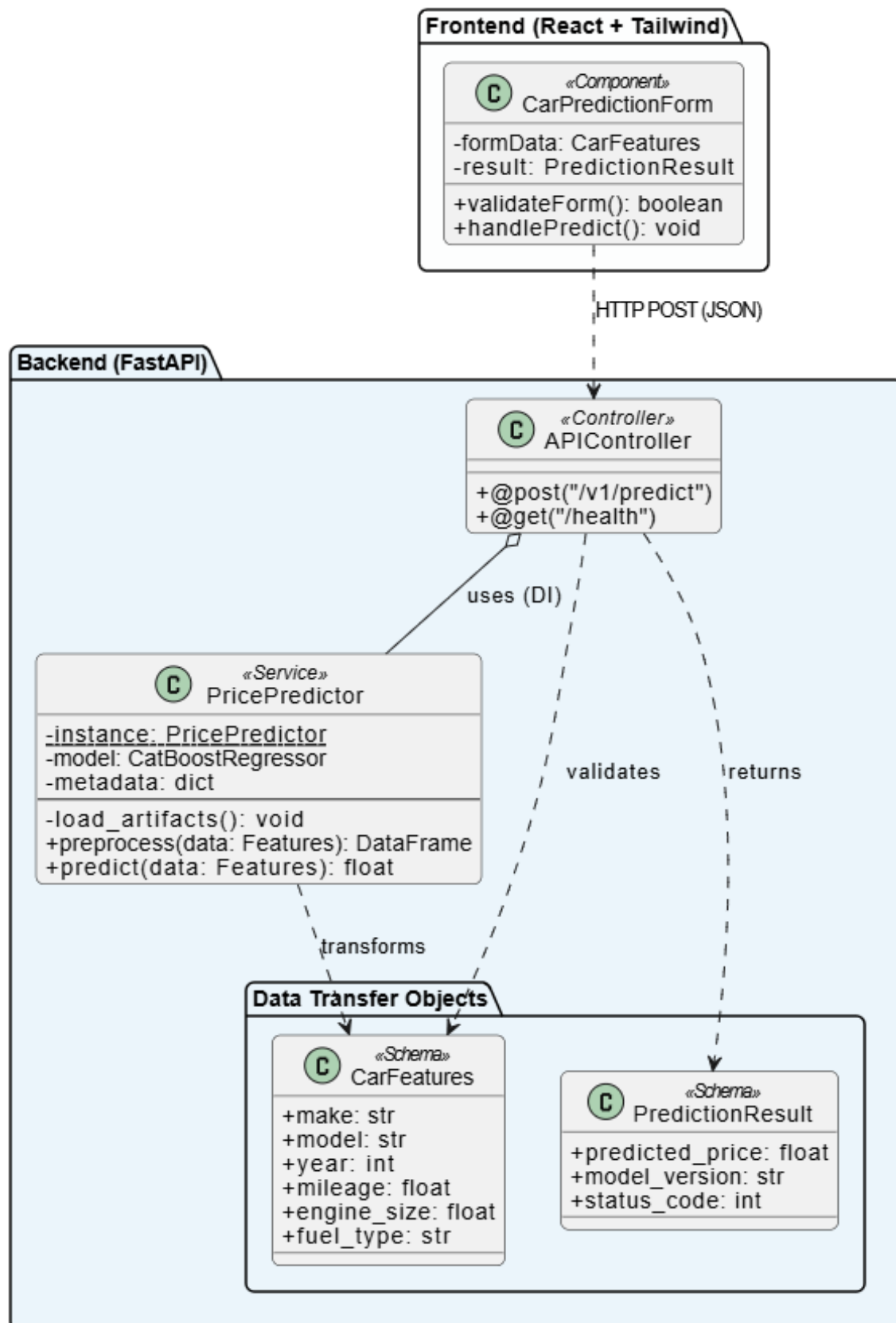
Case 2: Poor Performance

- R^2 below threshold
- System returns:
 - “Model performance insufficient”

Step 5: Deployment

- Best model becomes active
- System ready for prediction

4.5 Class Diagram



4.5.1 Overview

The class diagram represents the **structural design** of the Car Price Prediction System. It illustrates how different components (frontend, backend, and machine learning model) interact with each other.

The system follows a **layered architecture** consisting of:

- Frontend layer

Backend (API) layer

- Machine learning service layer
 - Data transfer objects (DTOs)
-

4.5.2 Frontend Layer

Class: CarPredictionForm

This component is responsible for:

- Collecting user input (car features)
- Validating form data
- Sending requests to the backend API
- Displaying prediction results

Attributes:

- formData: stores car input features
- result: stores prediction output

Methods:

- validateForm() → ensures valid input
 - handlePredict() → sends request to backend
-

4.5.3 Backend Layer (FastAPI)

Class: APIController

Acts as the main interface between frontend and backend.

Endpoints:

- POST /v1/predict → returns predicted price
- GET /health → checks system status
-

Responsibilities:

- Receive requests
- Validate input data
- Call prediction service
- Return response

4.5.4 Service Layer

Class: PricePredictor

This is the **core machine learning component**.

Attributes:

- model: trained **CatBoostRegressor**
- metadata: model-related information

Methods:

- load_artifacts() → load trained model
 - preprocess(data) → apply transformations
 - predict(data) → generate prediction
-

4.5.5 Data Transfer Objects (DTOs)

Class: CarFeatures

Represents input data from the user.

Attributes:

- make
- model
- year
- mileage
- engine_size
- fuel_type

Class: PredictionResult

Represents the system output.

Attributes:

- predicted_price
- model_version
- status_code

4.5.6 Relationships Between Classes

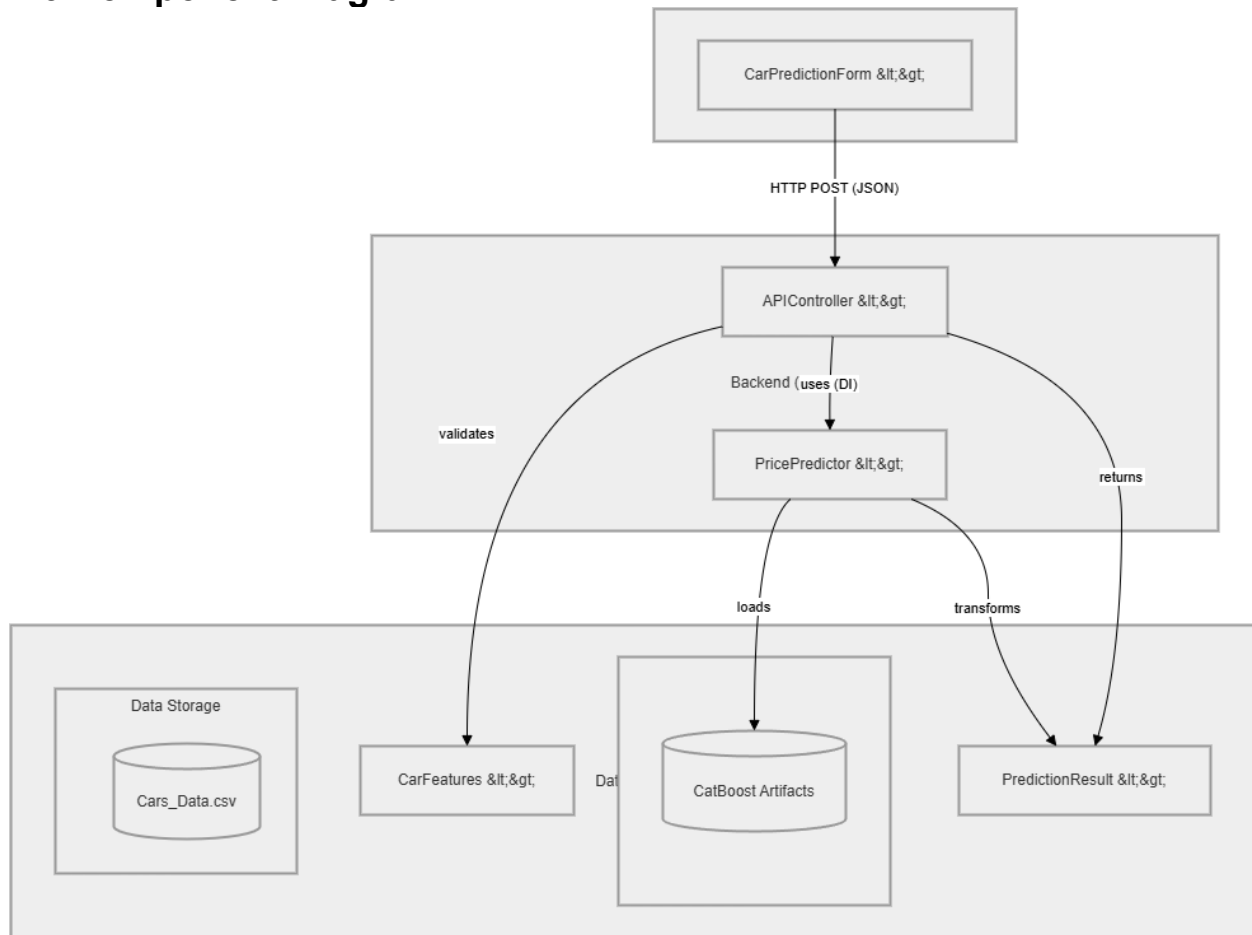
- **Frontend → Backend**
 - Communicates via HTTP POST (JSON)
 - **APIController → PricePredictor**
 - Uses dependency injection (DI)
 - **PricePredictor → DTOs**
 - Transforms input data
 - Returns prediction results
 - **Validation Flow**
 - Input is validated before prediction
 - Output is structured and returned
-

4.5.7 Architectural Insight (Important for viva)

This design ensures:

- Separation of concerns
- Scalability (frontend/backend independent)
- Reusability of ML model
- Clean API communication

4.6 Component Diagram



4.6.1 Overview

The component diagram represents the **high-level modular structure** of the system. It shows how different system components interact and exchange data.

4.6.2 Main Components

1. Frontend Component

- **CarPredictionForm (React)**
- Sends HTTP POST requests (JSON)

2. Backend API Component

- **ApiController (FastAPI)**
- Handles requests and validation
- Uses dependency injection (DI)

3. Machine Learning Component

PricePredictor

- Loads trained model
 - Applies preprocessing
 - Generates predictions
-

4. Data Components

- **CarFeatures (DTO)** → input data
 - **PredictionResult (DTO)** → output data
-

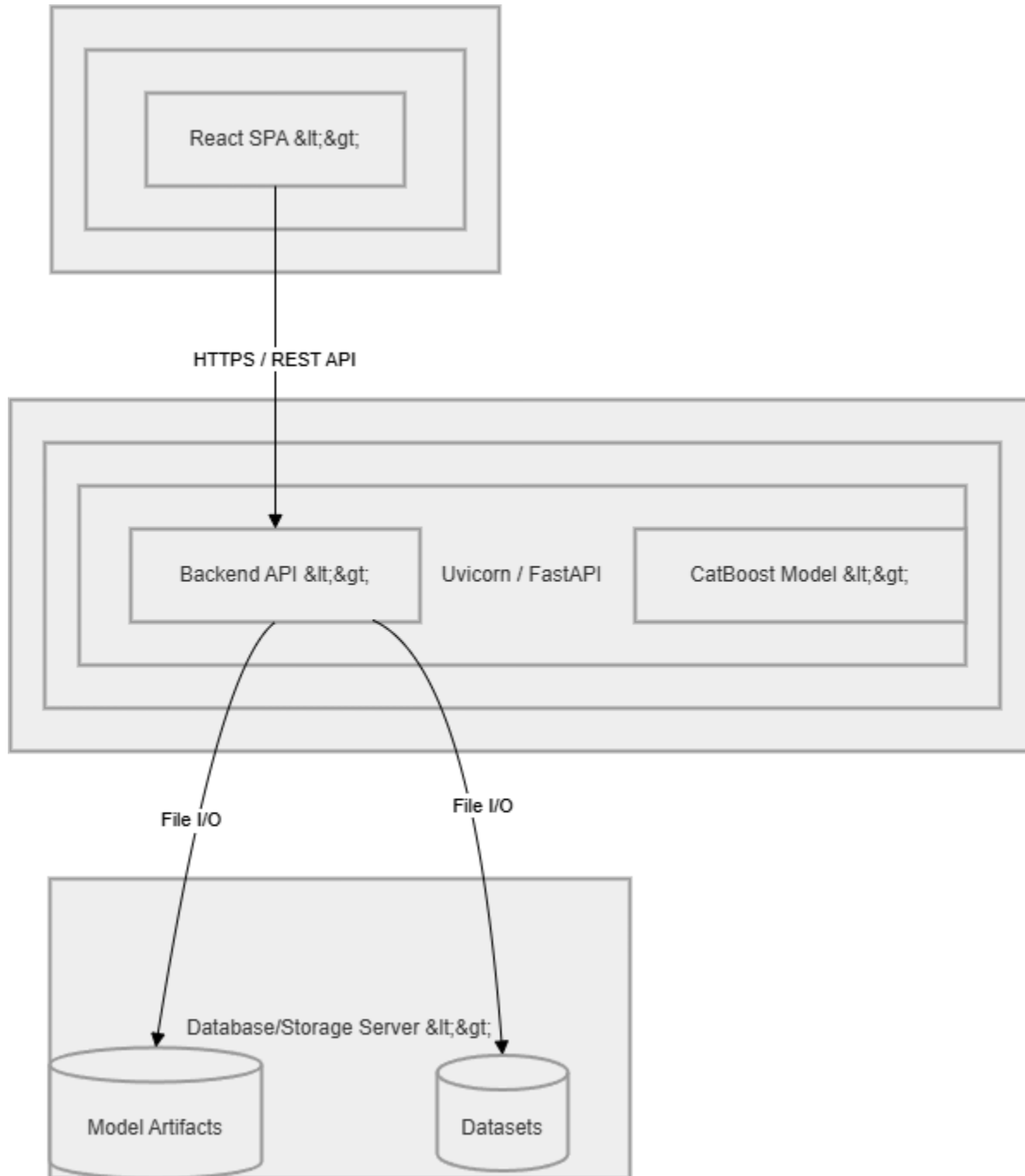
5. Storage Components

- **Cars_Data.csv** → dataset
 - **CatBoost Artifacts** → trained model
-

4.6.3 Interaction Flow

- Frontend → sends request → Backend
- Backend → validates → DTO
- Backend → calls → PricePredictor
- Model → loads artifacts → processes data
- Model → returns prediction → Backend
- Backend → sends result → Frontend

4.7 Deployment Diagram



4.7.1 Overview

The deployment diagram represents the **physical architecture** of the system, showing how components are deployed across different environments.

4.7.2 System Nodes

1. Client Layer

- **React SPA**
 - Runs in user browser
 - Sends requests via HTTPS
-

2. Server Layer

- **Backend API (FastAPI + Uvicorn)**
 - Handles API requests
 - Hosts machine learning logic
 - **CatBoost Model**
 - Loaded within backend environment
-

3. Storage Layer

- **Datasets**
 - Stored as files (CSV)
 - **Model Artifacts**
 - Saved trained models
-

4.7.3 Communication

- Client ↔ Server:
 - REST API (HTTPS)
- Server ↔ Storage:
 - File I/O operations

Chapter 4: Dataset Description

Chapter 5: Data Preprocessing

Chapter 7: Feature Engineering

Chapter 8: Model Building

Chapter 9: Model Evaluation

Chapter 10: Results & Discussion

Chapter 11: Deployment

Chapter 11.4: System Architecture

Chapter 13: UML System Design Diagrams